UrbanRoute — Tarjetas CRC corregidas y plan de commits

Oct 24, 2026 · @Gianmarco

Cambios respecto al avance (15/20)

Las tarjetas pasan de 16 a 18 clases y todas las colaboraciones son ahora recíprocas: 47 pares, cada uno nombrado en ambas tarjetas y convertido en una línea del diagrama de clases.

Nº	Cambio	Motivo
1	Colaboraciones recíprocas en todas las tarjetas, con responsabilidades pasivas ("Ser consultada por…", "Proveer sus datos a…")	Comentario 2 del profesor; se mantiene el máximo de 5 responsabilidades por tarjeta
2	`Estacion` nombra a `VentanaPrincipal`, `VisualizadorGrafo` y a todas las clases que la usan, y cada una la nombra de vuelta	Comentario 1 del profesor
3	Se elimina `NodoHash`: cada cubeta es una `ListaEnlazada<Estacion>`	Encadenamiento separado reutilizando la lista propia
4	Se agrega `Nodo<T>`	`ListaEnlazada`, `Cola` y `Pila` lo necesitan y no tenía tarjeta
5	Se elimina `SelectorArchivo`	`JFileChooser` es un componente de Swing que usa `VentanaPrincipal`; la tarjeta anterior saltaba a `ControladorRed`
6	`Grafo` contiene dos `TablaHash` (por ID y por nombre)	Coherencia garantizada en un solo lugar y delegación explícita (subcriterio 1.1)
7	Se agregan `Ruta` y `Subred` como resultados	El visualizador ya no llama a los algoritmos; solo dibuja resultados
8	BFS con `Cola`; `Pila` invierte el camino de Dijkstra	Basta un recorrido; la pila tiene un uso real
9	`Estacion` guarda su índice en el grafo; eliminar es un borrado real	El criterio 3.1 exige reflejar la baja en grafo, hash y archivo

Nº	Cambio	Motivo
10	`GestorArchivo` crea estaciones provisionales para IDs desconocidos	Los archivos del profesor cargan aunque el dataset esté incompleto
11	Las estructuras devuelven arreglos hacia afuera	Sin `java.util.Iterator` ni nodos internos expuestos
12	Nombres en formato Java y paquetes por capa	Tarjetas, diagrama y código coinciden letra por letra

Paquetes: `urbanroute.estructuras`, `urbanroute.modelo`, `urbanroute.logica`, `urbanroute.archivo`, `urbanroute.interfaz`.

Tarjetas CRC corregidas

Son 18 tarjetas en cinco paquetes; ninguna supera 5 responsabilidades y cada colaborador listado nombra a esta clase en su propia tarjeta.

Paquete estructuras

Nodo<T> (entidad auxiliar)

Responsabilidades	Colaboradores
Guardar un dato genérico y la referencia al siguiente nodo	—
Servir de eslabón a las estructuras enlazadas	ListaEnlazada, Cola, Pila

ListaEnlazada<T> (entidad auxiliar)

Responsabilidades	Colaboradores
Agregar, buscar y eliminar elementos encadenando nodos	Nodo
Informar su tamaño y copiarse a un arreglo para recorridos externos	—
Servir de cubeta para resolver colisiones	TablaHash
Servir de lista de adyacencia de cada estación	Grafo
Acumular las estaciones de una subred y las advertencias de carga	AnalizadorConectividad, GestorArchivo

Cola<T> (entidad auxiliar)

Responsabilidades	Colaboradores
Encolar y desencolar en orden FIFO en O(1)	Nodo
Proveer el orden de visita del BFS	AnalizadorConectividad

Pila<T> (entidad auxiliar)

Responsabilidades	Colaboradores
Apilar y desapilar en orden LIFO en O(1)	Nodo
Invertir el camino de Dijkstra (de destino→origen a origen→destino)	CalculadorRutas

ColaPrioridad (entidad auxiliar, min-heap sobre arreglos)

Responsabilidades	Colaboradores
Mantener pares (índice de estación, distancia) ordenados como min-heap	—
Insertar y extraer el mínimo en O(log n)	—
Entregar a Dijkstra la estación pendiente de menor distancia	CalculadorRutas

Paquete modelo

Estacion (entidad)

Responsabilidades	Colaboradores
Conocer su ID, nombre, línea, zona y capacidad de pasajeros por hora	—
Guardar la posición (índice) que el grafo le asigna	Grafo
Ser almacenada e indexada por su ID y por su nombre	TablaHash
Ser creada desde el dataset, como provisional o desde los datos del usuario	GestorArchivo, ControladorRed
Proveer sus datos para mostrarse, dibujarse, analizarse y agruparse	VentanaPrincipal, VisualizadorGrafo, AnalizadorHubs, AnalizadorConectividad, CalculadorRutas, Ruta, Subred

Conexion (entidad)

Responsabilidades	Colaboradores
Conocer el ID de origen, el ID de destino y el tiempo de traslado	—
Informar el extremo opuesto dado uno de sus extremos	—
Ser creada y almacenada en las listas de adyacencia	Grafo
Ser recorrida por los algoritmos de conectividad y rutas	AnalizadorConectividad, CalculadorRutas
Proveer sus datos para guardarse en archivo y dibujarse	GestorArchivo, VisualizadorGrafo

TablaHash (entidad, estructura propia)

Responsabilidades	Colaboradores
Calcular el índice con una función de dispersión propia sobre la clave (ID o nombre, según se construya)	—
Insertar una estación resolviendo colisiones por encadenamiento separado	ListaEnlazada, Estacion
Buscar y eliminar una estación por clave en O(1) promedio	ListaEnlazada, Estacion
Redimensionarse al superar el factor de carga 0,75 y listar todas sus estaciones	ListaEnlazada
Atender las consultas de estaciones que le delega el grafo	Grafo

Grafo (entidad, estructura propia, no dirigido y ponderado)

Responsabilidades	Colaboradores
Contener las dos tablas hash (por ID y por nombre) y mantenerlas coherentes con la adyacencia	TablaHash
Agregar o eliminar una estación con todas sus conexiones, asignándole un índice	Estacion, TablaHash, ListaEnlazada
Agregar o eliminar una conexión en ambas listas (no dirigido)	Conexion, ListaEnlazada
Entregar una estación por ID o nombre, sus vecinos, su grado y todas las estaciones o conexiones	TablaHash, Estacion, Conexion
Servir de fuente de datos para análisis, persistencia y dibujo	ControladorRed, AnalizadorConectividad, CalculadorRutas, AnalizadorHubs, GestorArchivo, VisualizadorGrafo

Ruta (entidad de resultado)

Responsabilidades	Colaboradores
Conocer las estaciones del trayecto en orden y el tiempo total	Estacion

Responsabilidades	Colaboradores
Indicar si existe ruta entre el origen y el destino	—
Ser creada por el algoritmo de Dijkstra	CalculadorRutas
Ser entregada, mostrada y resaltada	ControladorRed, VentanaPrincipal, VisualizadorGrafo

Subred (entidad de resultado)

Responsabilidades	Colaboradores
Conocer su número y las estaciones que la forman	Estacion
Informar su tamaño	—
Ser creada por el análisis de conectividad	AnalizadorConectividad
Ser entregada, listada y coloreada	ControladorRed, VentanaPrincipal, VisualizadorGrafo

Paquete logica

AnalizadorConectividad (control)

Responsabilidades	Colaboradores
Ejecutar BFS desde cada estación no visitada	Grafo, Cola, Conexion
Agrupar las estaciones alcanzadas en una subred	ListaEnlazada, Estacion, Subred
Detectar zonas incomunicadas (más de una subred tras un cierre)	—
Responder las solicitudes de análisis del coordinador	ControladorRed

CalculadorRutas (control)

Responsabilidades	Colaboradores
Verificar que el origen y el destino existan en la red	Grafo

Responsabilidades	Colaboradores
Ejecutar Dijkstra con una cola de prioridad	Grafo, ColaPrioridad, Conexion
Reconstruir el camino a partir de los predecesores	Pila, Estacion
Entregar la ruta con su tiempo total, o informar que no existe	Ruta
Responder las solicitudes del coordinador	ControladorRed

AnalizadorHubs (control)

Responsabilidades	Colaboradores
Calcular el grado de cada estación (centralidad de grado)	Grafo, Estacion
Ordenarlas de mayor a menor grado con un ordenamiento propio e identificar a todas las empatadas en el máximo	Estacion
Responder las solicitudes del coordinador	ControladorRed

ControladorRed (control, coordinador)

Responsabilidades	Colaboradores
Mantener la red en memoria y saber si hay cambios sin guardar	Grafo
Cargar y guardar la red a través del gestor de archivos	GestorArchivo
Agregar o eliminar estaciones y conexiones validando los datos del usuario	Grafo, Estacion
Solicitar los análisis y entregar sus resultados	AnalizadorConectividad, CalculadorRutas, AnalizadorHubs, Ruta, Subred
Atender las solicitudes de la interfaz	VentanaPrincipal

Paquete archivo

GestorArchivo (control)

Responsabilidades	Colaboradores
Leer el dataset y crear las estaciones en el grafo	Estacion, Grafo
Leer el archivo de grafo, crear las conexiones y las estaciones provisionales para IDs desconocidos	Grafo, Estacion
Validar cada línea y acumular advertencias sin detener la carga	ListaEnlazada
Escribir ambos archivos con el estado actual de la red	Grafo, Estacion, Conexion
Atender la carga por defecto, la carga nueva y el guardado	ControladorRed

Paquete interfaz

VentanaPrincipal (frontera, Swing con JFileChooser)

Responsabilidades	Colaboradores
Ofrecer los controles y diálogos y recoger los datos del usuario	ControladorRed
Mostrar alertas, errores y el resumen de advertencias de carga	ControladorRed
Mostrar la información de las estaciones y la búsqueda por nombre	Estacion
Mostrar los resultados de ruta, subredes y hubs	Ruta, Subred, Estacion
Contener el panel del grafo y pedirle redibujar o cambiar el modo de color	VisualizadorGrafo

VisualizadorGrafo (frontera, GraphStream solo para dibujar)

Responsabilidades	Colaboradores
Construir el dibujo a partir de la red en memoria	Grafo, Estacion, Conexion
Colorear por línea o por subred según el modo elegido	Estacion, Subred
Resaltar la ruta óptima y los hubs	Ruta, Estacion

Responsabilidades	Colaboradores
Redibujar tras cada cambio y entregar su panel a la ventana	VentanaPrincipal

Relaciones para el diagrama de clases

Cada uno de los 47 pares aparece en ambas tarjetas y será exactamente una línea del diagrama, con el tipo indicado abajo.

Tipo	Símbolo UML	Pares
Composición	◆	ListaEnlazada ◆ Nodo · Cola ◆ Nodo · Pila ◆ Nodo · TablaHash ◆ ListaEnlazada · Grafo ◆ TablaHash (×2: por ID y por nombre) · Grafo ◆ ListaEnlazada · Grafo ◆ Conexion · ControladorRed ◆ Grafo · VentanaPrincipal ◆ VisualizadorGrafo
Agregación	◇	TablaHash ◇ Estacion · Ruta ◇ Estacion · Subred ◇ Estacion
Asociación	→	ControladorRed → GestorArchivo · ControladorRed → AnalizadorConectividad · ControladorRed → CalculadorRutas · ControladorRed → AnalizadorHubs · VentanaPrincipal → ControladorRed · Grafo → Estacion
Dependencia (uso)	⇢	AnalizadorConectividad ⇢ Grafo, Cola, ListaEnlazada, Conexion, Estacion, Subred · CalculadorRutas ⇢ Grafo, ColaPrioridad, Pila, Conexion, Estacion, Ruta · AnalizadorHubs ⇢ Grafo, Estacion · GestorArchivo ⇢ Grafo, Estacion, Conexion, ListaEnlazada · ControladorRed ⇢ Estacion, Ruta, Subred · VentanaPrincipal ⇢ Estacion, Ruta, Subred · VisualizadorGrafo ⇢ Grafo, Estacion, Conexion, Ruta, Subred

Dinámica de colaboración

Ruta más rápida de PRO a PAL:

1. `VentanaPrincipal` recibe los IDs (o nombres) de origen y destino y los pasa a `ControladorRed`.

2. `ControladorRed` invoca a `CalculadorRutas`, que pide al `Grafo` las estaciones (el grafo delega en `TablaHash`) y sus vecinos.
3. `CalculadorRutas` usa `ColaPrioridad` para Dijkstra y `Pila` para reconstruir el camino, y devuelve una `Ruta`.
4. `VentanaPrincipal` muestra la ruta y pide a `VisualizadorGrafo` resaltarla.

Eliminar una estación:

1. `VentanaPrincipal` pide a `ControladorRed` eliminar la estación.
2. `ControladorRed` llama a `Grafo.eliminarEstacion(id)`, que quita las conexiones de los vecinos, libera el índice y la borra de ambas tablas hash.
3. `ControladorRed` pide a `AnalizadorConectividad` las nuevas subredes y las devuelve.
4. `VentanaPrincipal` muestra las subredes y pide a `VisualizadorGrafo` redibujar.

Formato de los archivos

Se usan los dos formatos del enunciado sin cambios; la robustez frente a los archivos del profesor está en cómo se leen.

Dataset (`dataset.csv`, estaciones):

```
ID_Estacion,Nombre,Linea,Zona,Capacidad_Pasajeros_Hora
PRO,Propatria,Linea 1,Catia,12000
```

Archivo de grafo (`grafo.csv`, conexiones no dirigidas):

```
Estacion_Origen,Estacion_Destino,Tiempo_Minutos
PRO,PER,2
```

Reglas de lectura

Situación	Comportamiento
Encabezado	Opcional; se detecta si la columna numérica de la primera línea no es un número
Codificación	UTF-8 sin BOM; si hay bytes inválidos se relee en Windows-1252 (protege la Ñ de CAÑ)
Separador	`,` o `;` (Excel en español)
Espacios y líneas vacías	Se recortan y se ignoran

Situación	Comportamiento
Columnas extra / faltantes	Las extra se ignoran; si faltan, la línea se descarta con advertencia
IDs	Se comparan sin espacios y en mayúsculas
ID o nombre repetido en el dataset	Se conserva el primero y se avisa
Capacidad no numérica o negativa	Línea descartada con advertencia
Tiempo ⩽ 0 o no numérico	Conexión descartada; se aceptan decimales como 2.5
Lazo (A,A,2)	Descartado con advertencia
Conexión repetida (A,B y luego B,A)	Se conserva la primera y se avisa
ID del grafo ausente en el dataset	Se crea una estación provisional (nombre = ID, línea y zona "Sin asignar", capacidad 0) y se avisa
Estaciones sin conexiones (LOS_D, CAP_L2, CSB)	Se cargan como subredes aisladas, sin error
Archivo vacío o ilegible	Se conserva la red actual y se muestra un error

La carga es atómica: se arma un grafo nuevo aparte y solo reemplaza al actual si la lectura terminó. Al final se muestra un resumen con el número de estaciones, conexiones y advertencias.

Flujo de carga y guardado

1. Si hay cambios sin guardar, una alerta ofrece Guardar, No guardar o Cancelar.
2. Un `JFileChooser` pide el dataset y otro el archivo de grafo, ambos filtrados a `.csv`. Si se cancela el dataset, se carga solo el grafo con estaciones provisionales.
3. "Actualizar repositorio" reescribe ambos archivos en sus rutas de origen, en UTF-8 y con encabezados estándar. Las estaciones van en orden de índice y cada conexión una sola vez.
4. Se escribe primero a un archivo temporal y luego se reemplaza el original, para no corromper datos si algo falla.
5. Al iniciar se cargan `data/dataset.csv` y `data/grafo.csv` con ruta relativa; si faltan, el programa arranca con la red vacía y un aviso.

Visualización

Un selector alterna entre colorear por línea (paleta asignada en orden de aparición; transbordos en gris) y por subred (un color por componente). En ambos modos la ruta óptima se resalta con trazo grueso y los hubs se dibujan más grandes.

Plan de commits

Son 28 commits entre el 25 de octubre y el 13 de noviembre, 14 de Gianmarco (G) y 14 de Alberto (A); del 10 al 13 de noviembre (4 de 21 días, 19 %) quedan reservados para pruebas, corrección y documentación.

Nº	Fecha	Autor	Mensaje del commit	Objetivo	Clases / archivos	Tiempo
1	Dom 25 oct	G	`feat: crear proyecto UrbanRoute con paquetes, datos maestros y GraphStream`	Proyecto NetBeans, paquetes, CSV maestros, jars en `lib/`, readme inicial; verificar GraphStream en JDK 26	`data/dataset.csv`, `data/grafo.csv`, `lib/*.jar`, `README.md`, `PruebaGraphStream`	2 h
2	Lun 26 oct	A	`feat: implementar Nodo y ListaEnlazada genérica`	Base de las estructuras: agregar, buscar, eliminar, tamaño, copia a arreglo	`Nodo`, `ListaEnlazada`, `PruebaLista`	2 h
3	Lun 26 oct	G	`feat: implementar Cola y Pila enlazadas`	FIFO para BFS y LIFO para rutas, en O(1)	`Cola`, `Pila`, `PruebaColaPila`	1,5 h
4	Mar 27 oct	A	`feat: implementar ColaPrioridad con min-heap`	Prioridad para Dijkstra en O(log n)	`ColaPrioridad`, `PruebaColaPrioridad`	1,5 h
5	Mar 27 oct	G	`feat: agregar entidades Estacion y Conexion`	Modelo de datos de la red	`Estacion`, `Conexion`	1,5 h
6	Mié 28 oct	A	`feat: implementar TablaHash con encadenamiento separado`	Dispersión propia, colisiones, redimensión a 0,75, modo ID o nombre	`TablaHash`, `PruebaTablaHash`	2 h

Nº	Fecha	Autor	Mensaje del commit	Objetivo	Clases / archivos	Tiempo
7	Mié 28 oct	G	`docs: agregar bocetos de interfaz y tarjetas CRC corregidas`	Bocetos a mano de ventana y diálogos (semana 5) y CRC corregidas	`docs/bocetos/*`, `docs/CRC.pdf`	1,5 h
8	Jue 29 oct	A	`feat: implementar Grafo no dirigido con lista de adyacencia`	Altas de estaciones y conexiones, vecinos, grado, búsqueda delegada a las hash	`Grafo`, `PruebaGrafo`	2 h
9	Vie 30 oct	G	`feat: eliminar estaciones y conexiones del grafo`	Bajas coherentes en adyacencia y ambas hash	`Grafo`, `PruebaGrafo`	2 h
10	Sáb 31 oct	A	`feat: detectar subredes con BFS`	Componentes conexos y zonas incomunicadas	`Subred`, `AnalizadorConectividad`, `PruebaConectividad`	2 h
11	Dom 1 nov	G	`feat: calcular ruta de menor tiempo con Dijkstra`	Ruta óptima o aviso de que no existe	`Ruta`, `CalculadorRutas`, `PruebaRutas`	2 h
12	Lun 2 nov	A	`feat: identificar hubs por centralidad de grado`	Ranking con ordenamiento propio y empates	`AnalizadorHubs`, `PruebaHubs`	1,5 h
13	Lun 2 nov	G	`feat: leer dataset y archivo de grafo con validación`	Encabezado, codificación, separador, advertencias, provisionales, carga atómica	`GestorArchivo`, `PruebaLectura`, `data/pruebas/*.csv`	2 h
14	Mar 3 nov	A	`feat: guardar la red en el dataset y el archivo de grafo`	Persistencia con archivo temporal y coherencia al recargar	`GestorArchivo`, `PruebaEscritura`	2 h
15	Mié 4 nov	G	`feat: implementar ControladorRed`	Coordinación, cambios sin guardar, carga del maestro	`ControladorRed`	2 h

Nº	Fecha	Autor	Mensaje del commit	Objetivo	Clases / archivos	Tiempo
16	Jue 5 nov	A	`feat: diseñar VentanaPrincipal y conectar carga y guardado`	Ventana en el GUI Builder, JFileChooser y alerta antes de cargar	`VentanaPrincipal` (.java y .form)	2 h
17	Jue 5 nov	G	`feat: dibujar la red con GraphStream`	Panel del grafo que se redibuja tras cada cambio	`VisualizadorGrafo`, `VentanaPrincipal`	2 h
18	Vie 6 nov	A	`feat: colorear la red por línea o por subred`	Selector de modo y paleta dinámica	`VisualizadorGrafo`, `VentanaPrincipal`	1,5 h
19	Sáb 7 nov	G	`feat: agregar diálogos para modificar la red`	Agregar estación o conexión y eliminar estación o conexión, con validación	`VentanaPrincipal`, `ControladorRed`	2 h
20	Dom 8 nov	A	`feat: mostrar ruta óptima, subredes y hubs`	Paneles de resultados y resaltado en el grafo	`VentanaPrincipal`, `VisualizadorGrafo`	2 h
21	Lun 9 nov	G	`feat: buscar estaciones por nombre y mostrar sus detalles`	Tabla por nombre, barra de estado y ayuda al usuario	`VentanaPrincipal`, `ControladorRed`	1,5 h
22	Lun 9 nov	A	`feat: cerrar y reabrir estaciones temporalmente`	Extra opcional para puntos adicionales; se omite si hay atraso	`Estacion`, `ControladorRed`, `VentanaPrincipal`	1,5 h
23	Mar 10 nov	G	`test: agregar archivos de prueba con errores y casos límite`	Archivos vacíos, mal formados, IDs desconocidos, pesos inválidos, red desconectada	`data/pruebas/*`, `docs/casos-de-prueba.md`	2 h
24	Mié 11 nov	A	`fix: corregir errores encontrados en las pruebas`	Depende de lo que revele el commit 23	Por definir	2 h
25	Jue 12 nov	G	`docs: completar Javadoc de todas las clases`	Javadoc completo y generado sin advertencias	Todas	2 h

Nº	Fecha	Autor	Mensaje del commit	Objetivo	Clases / archivos	Tiempo
26	Jue 12 nov	A	`docs: agregar diagrama de clases final`	Diagrama coherente con las CRC: 47 pares y sus tipos	`docs/diagrama-clases.*`	2 h
27	Vie 13 nov	G	`docs: completar readme`	Descripción, integrantes, URL del repositorio e instrucciones	`README.md`	1 h
28	Vie 13 nov	A	`refactor: eliminar paquete de pruebas y verificar entrega`	Proyecto limpio; el ZIP compila en otra máquina	`pruebas.*`	1,5 h
—	Sáb 14 nov	Ambos	Sin commit	Generar el ZIP con `lib/`, subirlo al aula y confirmar que el repositorio es público	—	1 h

Reglas de trabajo

- Las clases del paquete `urbanroute.pruebas` muestran sus resultados con `JOptionPane`, sin consola, y se eliminan en el commit 28.
- Cada uno hace sus commits desde su propia cuenta de GitHub: `git pull` antes de empezar y `git push` al terminar.
- Cada integrante revisa lo que sube el otro, porque en la defensa pueden preguntarle por cualquier commit.
- En los commits de interfaz (16 a 21) se entrega la lista de componentes y el código de los eventos; la ventana se diseña a mano en el GUI Builder.
- Pendiente de confirmar: la entrega de bocetos se asumió para el jueves 29 de octubre.